

El lenguaje y ecosistema ideal para el cálculo numérico, simbólico y la visualización de datos.



Lenguaje de programación potente, fácil de aprender y ampliamente usado en ciencia e ingeniería.

NumPy · SciPy · Matplotlib · SymPy

¿QUÉ ES PYTHON?

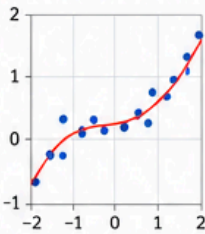
Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, con una sintaxis clara y legible.

Está diseñado para ser **fácil de usar** y muy flexible.

Es **gratuito, de código abierto** y cuenta con una **enorme comunidad**.

¿PARA QUÉ SIRVE?

- ✓ Cálculos numéricos y científicos
- ✓ Álgebra lineal y métodos numéricos
- ✓ Análisis y manipulación de datos
- ✓ Visualización de datos
- ✓ Simulación y modelado
- ✓ Inteligencia artificial y machine learning
- ✓ Automatización y desarrollo web



VENTAJAS

Gratuito y de código abierto

Sintaxis simple y legible

Multiplataforma (Windows, Linux, macOS)

Gran comunidad y abundante documentación

Ecosistema científico muy completo y potente

Muy usado en investigación e industria

ECOSISTEMA CIENTÍFICO

NumPy Arreglos n-dimensionales eficientes y álgebra lineal básica.

SciPy Algoritmos avanzados para optimización, integración, EDO, interpolación y más.

Matplotlib Gráficos 2D de alta calidad y personalizables.

SymPy Cálculo simbólico: álgebra, derivadas, integrales, ecuaciones, simplificación.

En conjunto permiten resolver problemas complejos de forma eficiente.

FORMAS DE TRABAJAR

Puedes usar Python de varias maneras según tus necesidades.

1. CONSOLA INTERACTIVA

Escribe comandos línea por línea y ve resultados al instante.

```
>>> 2 + 3
5
>>> import numpy as np
>>> np.arange(5)
array([0, 1, 2, 3, 4])
>>> |
```

2. SCRIPTS DE PYTHON

Escribe tu código en archivos con extensión .py y ejecútalos.

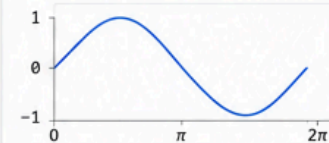
```
@mi_script.py
1 import numpy as np
2 A = np.eye(3)
3 b = np.array([1, 2, 3])
4 x = np.linalg.solve(A, b)
5 print(x)
```

```
$ python mi_script.py
[1. 2. 3.]
```

3. JUPYTER NOTEBOOK

Entorno interactivo que combina código, texto, ecuaciones y gráficos.

```
In [1]: import numpy as np
In [2]: x = np.linspace(0, 2*np.pi, 100)
In [3]: import matplotlib.pyplot as plt
In [4]: plt.plot(x, np.sin(x))
Out[4]: [<matplotlib.lines.Line2D at 0x...>]
```



4. EDITORES DE CÓDIGO

Entornos avanzados como VS Code o Spyder con depuración y autocompletado.

Visual Studio Code

spyder

Consejo: Si estás comenzando, **Jupyter Notebook** es excelente para explorar y aprender.

PRIMEROS PASOS

- 1 Ejecuta Python desde la consola escribiendo: **python**
- 2 Escribe una instrucción y presiona Enter.
- 3 Python mostrará el resultado de la expresión.
- 4 Usa **help()** para obtener ayuda y **dir()** para ver funciones disponibles.

Ejemplos en la consola de Python

```
>>> print("Hola, Python científico!")
Hola, Python científico!
>>> 2 + 3*4
14
>>> type(3.14)
<class 'float'>
>>> help(len)
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

EJEMPLO RÁPIDO

```
# Calcula la solución de un sistema lineal
# usando NumPy
import numpy as np
A = np.array([[3, 1, -1],
              [2, 4, 1],
              [-1, 2, 5]])
b = np.array([4, 1, 1])
x = np.linalg.solve(A, b)
print("Solución x =")
print(x)
# Resultado:
# Solución x =
# [1. 1. 1.]
```

¡Escribe, explora, experimenta y resuelve!



CONSEJOS

- Python distingue mayúsculas y minúsculas.
- La indentación (espacios) es obligatoria.
- Los comentarios comienzan con #.
- Usa nombres de variables descriptivos.

NOTA IMPORTANTE

Python es el lenguaje base.
NumPy, SciPy, Matplotlib y SymPy son bibliotecas que se importan cuando se necesitan.

MÁS INFORMACIÓN

Documentación oficial de Python:

<https://docs.python.org/>

Python Software Foundation: python.org





1 VARIABLES BÁSICAS

Python tiene varios tipos de datos integrados.

Tipo	Ejemplo	Descripción
int	x = 5	Entero
float	y = 3.1416	Número decimal
complex	z = 2 + 3j	Número complejo
bool	b = True	Verdadero o falso
str	s = "Hola"	Cadena de texto

Ejemplos

```
x = 10          # entero
y = 2.5         # flotante
z = 1 + 4j      # complejo
b = (x > 5)     # booleano (True)
s = 'Python'    # cadena de texto
```



Python distingue mayúsculas y minúsculas:
Variable, variable y VARIABLE son diferentes.

2 LISTAS DE PYTHON

Las listas son estructuras dinámicas que pueden contener elementos de distintos tipos.

```
L = [1, 2, 3, 4]          # lista de enteros
mix = [1, 'a', 3.5, True] # tipos mixtos
L[0]                     # primer elemento (1)
L[-1]                    # último elemento (4)
L[1:3]                   # porción [2, 3]
L.append(5)              # agrega al final -> [1,2,3,4,5]
L.pop()                  # elimina último -> [1,2,3,4]
len(L)                   # longitud de la lista (4)
```

LISTA vs ARREGLO (diferencias clave)

Lista (Python)	Arreglo (NumPy)
Puede contener distintos tipos	Todos los elementos del mismo tipo
Operaciones más lentas (bucle por elemento)	Operaciones vectorizadas (muy rápidas)
Menos memoria eficiente	Más memoria eficiente
No tiene forma (shape)	Tiene forma (shape) y dimensiones



Para cálculo numérico y científico es mejor usar arreglos de NumPy.

3 IMPORTAR NUMPY

NumPy es la biblioteca fundamental para trabajar con arreglos numéricos.

```
import numpy as np
```



Proporciona estructuras de datos eficientes (arrays) y funciones matemáticas de alto nivel.

CONVERSIÓN ENTRE LISTA Y ARREGLO

```
L = [1, 2, 3]
a = np.array(L) # lista -> arreglo
a.tolist()      # arreglo -> lista
```



Usa `np.array()` para convertir listas en arreglos de NumPy.

4 CREAR ARREGLOS CON NUMPY

Un arreglo (array) es una estructura de datos n-dimensional eficiente para almacenar números del mismo tipo.

DESDE LISTAS O TUPLAS

```
v = np.array([1, 2, 3])
A = np.array([[1, 2, 3],
               [4, 5, 6]])
```

CEROS, UNOS E IDENTIDAD

```
Z = np.zeros((2, 3)) # matriz 2x3 de ceros
O = np.ones((3, 2))  # matriz 3x2 de unos
I = np.eye(3)         # identidad 3x3
```

RANGOS Y ESPACIADOS

```
r1 = np.arange(0, 10, 2) # [0 2 4 6 8]
r2 = np.linspace(0, 1, 5) # 5 puntos 0..1
r3 = np.logspace(-2, 2, 5) # 10^-2 a 10^2
```

VALORES ALEATORIOS

```
rd = np.random.rand(2, 3) # uniformes [0,1)
rn = np.random.randn(2, 3) # normales N(0,1)
ri = np.random.randint(1, 11, (2, 3)) # enteros 1..10
```

★ Puedes fijar semilla para reproducibilidad:
`np.random.seed(0)`

ARREGLO VACÍO

```
vacio = np.empty((2, 3)) # contiene valores arbitrarios
# (no está inicializado)
```



Los índices en Python/NumPy comienzan en 0. El primero es 0, el segundo es 1, ...

6 INDEXACIÓN Y REBANADO (SLICING)

Accede a elementos y subarreglos.

```
A = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

	0	1	2	← columnas
0	1	2	3	
1	4	5	6	
2	7	8	9	
↓ filas				

Ejemplos

```
A[0, 0] # 1 (fila 0, columna 0)
A[1, 2] # 6
A[0, :] # [1 2 3] fila 0, todas columnas
A[:, 1] # [2 5 8] todas filas, columna 1
A[0:2, 1:3] # [[2 3]
               [5 6]] pasos de 2
A[:, 2:] # [[1 3]
             [7 9]] última fila
A[-1, :] # [7 8 9] última fila
A[:, -1] # [3 6 9] última columna
```

7 OPERACIONES BÁSICAS CON ARREGLOS

Las operaciones se aplican elemento a elemento (vectorizadas).

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b # [5 7 9]
a - b # [-3 -3 -3]
a * b # [4 10 18]
a / b # [0.25 0.4 0.5]
-a    # [-1 -2 -3]
np.sqrt(a) # [1. 1.414... 1.732...]
np.sum(a) # 6 (suma de elementos)
np.mean(a) # 2.0 (promedio)
np.max(a) # 3 (máximo)
np.min(a) # 1 (mínimo)
```

¡ATENCIÓN!

$A * B$ es producto elemento a elemento.
Para producto matricial usa `@` o `np.dot(A, B)`.

Ejemplo:

```
A = np.array([[1, 2],
               [3, 4]])
B = np.array([[5, 6],
               [7, 8]])

A * B # [[ 5 12]
         [ 21 32]]
A @ B # [[19 22]
         [43 50]]
```

8 BUENAS PRÁCTICAS

- ✓ Usa NumPy para cálculos numéricos (no listas).
- ✓ Verifica las formas (shape) antes de operar.
- ✓ Aprovecha la vectorización (evita bucles).
- ✓ Usa nombres claros y descriptivos.
- ✓ Fija semilla en aleatorios para reproducibilidad.
`np.random.seed(42)`

9 ERRORES FRECUENTES

- ✗ Usar `*` en lugar de `@` para producto matricial.
- ✗ Olvidar que los índices empiezan en 0.
- ✗ Formas incompatibles para operar arreglos.
- ✗ Crear arreglos muy grandes sin necesidad (memoria).
- ✗ Confundir lista de Python con arreglo de NumPy.

★ RECUERDA

- Lista (Python): flexible, mezclas tipos, más lenta.
- Arreglo (NumPy): mismo tipo, rápido y eficiente.
- Importa NumPy siempre como `np`.
- Es la base para SciPy, Matplotlib y análisis científico.

```
import numpy as np
```





1 CREAR VECTORES (1D)

Arreglos unidimensionales.

```
import numpy as np

v = np.array([1, 2, 3, 4]) # desde lista
w = np.arange(0, 5)        # 0 1 2 3 4
x = np.linspace(0, 1, 6)   # 6 puntos entre 0 y 1
y = np.logspace(-1, 1, 3)  # 10^-1, 10^0, 10^1
z = np.zeros(5)            # ceros
o = np.ones(4)             # unos
r = np.random.rand(5)      # aleatorios [0,1)
```

Ejemplo

```
v = [1 2 3 4]
w = [0 1 2 3 4]
x = [0. 0.2 0.4 0.6 0.8 1. ]
y = [ 0.1 1. 10. ]
r = [0.37 0.82 0.11 0.56 0.93] # ejemplo
```



TIP: Los índices en Python/NumPy comienzan en 0.

2 CREAR MATRICES (2D)

Arreglos bidimensionales.

```
A = np.array([[1, 2, 3],          # desde lista
              [4, 5, 6]])
Z = np.zeros((3, 4))             # 3 filas x 4 columnas
O = np.ones((2, 3))             # 2 x 3 de unos
I = np.eye(3)                   # identidad 3x3
R = np.random.rand(2, 3)        # aleatorios [0,1)
N = np.random.randint(1, 10, (2,3)) # enteros 1..9
```

Ejemplo

```
A =
[[1 2 3]
 [4 5 6]]

Z =
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

I =
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```



`np.eye(n)` crea la matriz identidad $n \times n$.

3 INFORMACIÓN DE UN ARREGLO

Atributos útiles.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

A.shape      # (2, 3)  filas, columnas
A.ndim       # 2       número de dimensiones
A.size       # 6       total de elementos
A.dtype      # int64    tipo de datos
A.T          # transpuesta
A.ravel()    # convierte en vector 1D (copia)
A.flatten()  # convierte en vector 1D (copia)
```

Ejemplo

```
A.shape -> (2, 3)
A.ndim  -> 2
A.size  -> 6
A.dtype -> int64
A.T      [1  4]
          [2  5]
          [3  6]
```



Usa `A.T` para la transpuesta (para matrices 2D).

4 INDEXACIÓN Y REBANADO (SLICING)

Accede a elementos, filas, columnas y submatrices.

```
A = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])
```

0 1 2 3 ← columnas

```

0  1  2  3  4
1  5  6  7  8
2  9 10 11 12
↓
filas
```

Selección	Código	Resultado
Elemento (fila 1, col 2)	<code>A[1, 2]</code>	7
Primera fila	<code>A[0, :]</code>	[1 2 3 4]
Segunda columna	<code>A[:, 1]</code>	[2 6 10]
Filas 0 a 1, todas columnas	<code>A[0:2, :]</code>	[[1 2 3 4] [5 6 7 8]]
Todas filas, columnas 1 a 2	<code>A[:, 1:3]</code>	[[2 3] [6 7] [10 11]]
Última fila	<code>A[-1, :]</code>	[9 10 11 12]
Última columna	<code>A[:, -1]</code>	[4 8 12]
Paso 2 en filas	<code>A[::2, :]</code>	[[1 2 3 4] [9 10 11 12]]



El formato es: `A[fila, columna]`. Usa `:` para seleccionar todo en esa dimensión.

5 OPERACIONES ELEMENTO A ELEMENTO

Las operaciones actúan sobre cada elemento.

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
```

```
A + B      # suma
A - B      # resta
A * B      # multiplicación elemento a elemento
A / B      # división elemento a elemento
A ** 2     # potencia elemento a elemento
np.sqrt(A) # raíz cuadrada
np.sin(A)  # funciones universales (ufuncs)
```

Ejemplo

```
A + B =
[[ 6  8]
 [10 12]]

A * B =
[[ 5 12]
 [21 32]]

A ** 2 =
[[ 1  4]
 [ 9 16]]

np.sqrt(A) =
[[ 1.  1.41421356]
 [ 1.73205081  2.        ]]
```



¡ATENCIÓN! `A * B` es elemento a elemento, **NO** producto matricial.

6 PRODUCTO MATRICIAL

Usa `@` (operador matmul) o `np.dot` para el producto matricial.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]]) # 2x3
B = np.array([[7, 8],
              [9, 10],
              [11, 12]]) # 3x2

C1 = A @ B # recomendado
C2 = np.dot(A, B) # equivalente
```

Ejemplo

```
C = A @ B =
[[58 64]
 [139 154]]
```



Las dimensiones internas deben coincidir (columnas de A = filas de B).

7 MULTIPLICACIÓN POR ESCALAR

Cada elemento se multiplica por el escalar.

```
A = np.array([[1, 2],
              [3, 4]])
2 * A # o A * 2
```

Ejemplo

```
2 * A =
[[2 4]
 [6 8]]
```



Funciona con `+`, `-`, `*`, `/` y potencias.

8 FUNCIONES Y UTILIDADES FRECUENTES

```
np.sum(A, axis=0) # suma por columnas
np.sum(A, axis=1) # suma por filas
np.mean(A, axis=0) # promedio por columnas
np.max(A) # máximo global
np.min(A, axis=1) # mínimo por fila
np.abs(A) # valor absoluto
np.round(A, 2) # redondeo a 2 decimales
np.transpose(A) # transpuesta (equiv. a A.T)
np.concatenate((u,v)) # concatena vectores
np.vstack((A,B)) # apila filas (vertical)
np.hstack((A,B)) # apila columnas (horizontal)
```



Explora `dir(np)` para ver todas las funciones disponibles.

9 BUENAS PRÁCTICAS

- ✓ Usa NumPy en lugar de listas para cálculos numéricos.
- ✓ Verifica las dimensiones (shape) antes de operar.
- ✓ Usa `@` para producto matricial.
- ✓ Aprovecha las funciones vectorizadas (más rápidas).
- ✓ Evita bucles cuando una operación vectorizada sea posible.

10 ERRORES FRECUENTES

- ✗ Confundir `A * B` (elemento a elemento) con `A @ B`.
- ✗ Olvidar que los índices comienzan en 0.
- ✗ Omitir los paréntesis en `np.arange(limite)`.
- ✗ Dimensiones incompatibles en operaciones.
- ✗ Modificar un rebanado puede afectar al original.

★ RECUERDA

- Los arreglos de NumPy son eficientes y rápidos.
- Soportan operaciones matemáticas completas.
- Base fundamental para el cálculo científico en Python.



**1 CONDICIONAL IF - ELIF - ELSE**

Permite ejecutar bloques de código según se cumplan condiciones.

SINTAXIS

```
if condición:
    instrucciones

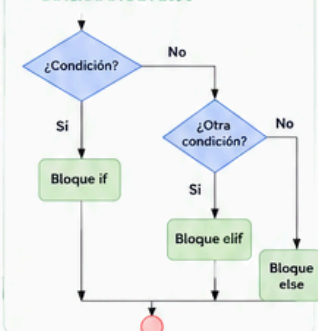
elif otra_condición:
    instrucciones

else:
    instrucciones
```

EJEMPLO

```
x = 7
if x > 0:
    print('x es positivo')      # condición verdadera
elif x < 0:
    print('x es negativo')     # condición falsa
else:
    print('x es cero')         # condición falsa
```

TIP: La indentación (espacios o tabulaciones) es obligatoria en Python.

DIAGRAMA DE FLUJO**2 OPERADORES DE COMPARACIÓN Y LÓGICOS****COMPARACIÓN**

```
== igual que
!= diferente de
< menor que
> mayor que
<= menor o igual que
>= mayor o igual que
```

LÓGICOS

```
and y (ambas verdaderas)
or o (al menos una verdadera)
not no (niega la condición)
```

EJEMPLOS

```
a, b = 5, 10
a < b and a != 0      # True
a > b or b == 10      # True
not (a == b)          # True
a >= 5 and b <= 10    # True
```

TIP: Los operadores lógicos actúan sobre expresiones booleanas.

3 SELECCIÓN MÚLTIPLE: MATCH - CASE (Python 3.10+)

Se usa cuando existen varias opciones posibles.

SINTAXIS

```
match variable:
    case valor1:
        instrucciones
    case valor2:
        instrucciones
    case _: # por defecto
        instrucciones
```

EJEMPLO

```
dia = 3
match dia:
    case 1:
        print('Lunes')
    case 2:
        print('Martes')
    case 3:
        print('Miércoles')
    case 4:
        print('Jueves')
    case _:
        print('Otro día')
```

TIP: El guion bajo _ captura cualquier valor que no coincida con los anteriores.

4 CICLO FOR

Se usa cuando se conoce el número de iteraciones.

SINTAXIS

```
for variable in iterable:
    instrucciones
```

EJEMPLOS**1 Contar del 1 al 5**

```
for k in range(1, 6):
    print(k)      # 1 2 3 4 5
```

2 Recorrer una lista

```
colores = ['rojo', 'verde', 'azul']
for c in colores:
    print(c)      # rojo verde azul
```

TIP: **range**(inicio, fin, paso)
El valor fin NO se incluye.

5 CICLO WHILE

Se ejecuta mientras la condición sea verdadera.

SINTAXIS

```
while condición:
    instrucciones
```

EJEMPLO: Factorial de n

```
n = 5
fact = 1
k = 1
while k <= n:
    fact = fact * k
    k = k + 1
print(fact)      # 120
```

Calcula $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$
mientras k sea
menor o igual que n.

TIP: Asegúrate de que la condición cambie en cada iteración para evitar ciclos infinitos.

6 BREAK

Sale inmediatamente del ciclo.

EJEMPLO

```
for k in range(1, 11):
    if k == 6:
        break      # sale cuando k = 6
    print(k)
# 1 2 3 4 5
```

7 CONTINUE

Salta a la siguiente iteración del ciclo.

EJEMPLO

```
for k in range(1, 10):
    if k % 2 == 0:
        continue   # salta los pares
    print(k)
# 1 3 5 7 9
```

Útil para omitir ciertas iteraciones sin salir del ciclo.

8 FUNCIONES ÚTILES PARA CICLOS**range(inicio, fin, paso)**

Genera una secuencia de números.

```
range(0, 10, 2)      # 0, 2, 4, 6, 8
```

enumerate(iterable)

Devuelve índice y valor al recorrer.

```
for i, v in enumerate(['a', 'b', 'c']):
    print(i, v)      # (0, 'a') (1, 'b') (2, 'c')
```

zip(iter1, iter2, ...)

Itera varios iterables en paralelo.

```
for a, b in zip([1, 2, 3], ['x', 'y', 'z']):
    print(a, b)      # (1, 'x') (2, 'y') (3, 'z')
```

Estas herramientas facilitan y hacen más legible tu código.

9 ERRORES FRECUENTES

- Olvidar los dos puntos : al final de if, for, while, etc.
- Indentación incorrecta.
Usa 4 espacios (recomendado).
- Confundir = (asignación) con == (comparación).
Ejemplo: if x = 5 es ERROR.
- Pensar que el último valor de range está incluido.
range(1, 5) -> 1, 2, 3, 4
- No actualizar variables en un while, causando ciclos infinitos.
- Usar continue cuando se quiere salir del ciclo.
Usa break.

10 BUENAS PRÁCTICAS

- Usa nombres descriptivos para variables y bucles.
- Mantén el código bien indentado y estructurado.
- Comenta tu código para explicar la lógica.
- Evita cálculos innecesarios dentro de los ciclos.
- Usa break y continue con moderación.
- Prueba tu código con casos simples primero.

Un código claro y estructurado es más fácil de entender, depurar y mantener.

Las estructuras de control son la base de cualquier programa.

if, for, while, break, continue, range, match
→ Control total sobre tu programa.





1 ¿QUÉ ES UN SCRIPT?

Un script es un archivo de texto con instrucciones que Python ejecuta en orden.

- ✓ Tiene extensión .py
- ✓ Se ejecuta desde la consola o un editor.
- ✓ Puede importar librerías y otros módulos.
- ✓ Permite definir funciones y reutilizarlas.
- ✓ Ideal para programas completos y reproducibles.

Ejemplo: archivo `saludo.py`

```
1 # saludo.py
2 def saludar(nombre):
3     print(f"Hola, {nombre}!")
4
5 if __name__ == "__main__":
6     saludar("Mundo")
```

TIP: Ejecuta un script desde la consola con:
`python saludo.py`

2 DEFINIR UNA FUNCIÓN

Una función realiza una tarea específica y puede recibir datos de entrada y devolver resultados.

```
def nombre_funcion(param1, param2, ...):
    """Docstring opcional: describe la función."""
    # instrucciones
    return resultado
```

Ejemplo: función que calcula el área de un círculo

```
def area_circulo(r):
    """Devuelve el área de un círculo de radio r."""
    import numpy as np
    return np.pi * r**2
```

Llamada a la función

```
a = area_circulo(3)
print(a) # 28.274333882398138
```

TIP: Usa nombres descriptivos y en minúsculas con guiones bajos (snake_case).

3 PARÁMETROS Y ARGUMENTOS

Concepto	Descripción
Parámetro	Variable definida en la función.
Argumento	Valor que se pasa a la función cuando se llama.
Posicional	Los argumentos se pasan en el mismo orden que los parámetros.
Nombrado (keyword)	Se especifica el nombre del parámetro al llamar la función.
Valores por defecto	La función usa un valor si no se proporciona un argumento.

Ejemplo

```
def potencia(x, n=2): # n tiene valor por defecto 2
    return x**n

potencia(3) # 9 -> usa n = 2
potencia(3, 3) # 27 -> argumento posicional
potencia(x=3, n=4) # 81 -> argumentos nombrados
```

TIP: Los valores por defecto deben ir al final de la lista de parámetros.

4 VALOR DE RETORNO

Una función puede devolver uno o más valores usando la palabra clave `return`.

Ejemplo: retorna múltiples valores

```
def raices_cuadraticas(a, b, c):
    import numpy as np
    d = b**2 - 4*a*c
    x1 = (-b + np.sqrt(d)) / (2*a)
    x2 = (-b - np.sqrt(d)) / (2*a)
    return x1, x2
```

Uso

```
r1, r2 = raices_cuadraticas(1, -3, 2)
print(r1, r2) # 2.0 1.0
```

TIP: Si la función no usa `return`, devuelve `None`.

5 VARIABLES LOCALES Y GLOBALES

Las variables creadas dentro de una función son locales. Las globales se definen fuera de la función.

```
x = 10 # variable global

def muestra():
    y = 5 # variable local
    print(x) # accede a la global
    print(y)

muestra()
# print(y) # Error: y no existe fuera de la función
```

Palabra clave `global`

Permite modificar una variable global dentro de la función.

```
contador = 0
def aumentar():
    global contador
    contador += 1
aumentar(): aumentar()
print(contador) # 2
```

TIP: Evita usar `global` salvo que sea realmente necesario.

6 DOCUMENTACIÓN: DOCSTRINGS

Usa docstrings para documentar tus funciones.

```
def suma(a, b):
    """
    Devuelve la suma de a y b.
    Parámetros
    -----
    a : número
    b : número
    Retorna
    -----
    int o float
    """
    return a + b
```

Accede a la documentación

```
help(suma)
print(suma.__doc__)
```

TIP: Una buena documentación facilita el uso y mantenimiento.

7 FUNCIONES ANÓNIMAS (LAMBDA)

Funciones pequeñas sin nombre usando la palabra clave `lambda`.

```
cuadrado = lambda x: x**2
print(cuadrado(5)) # 25
```

Ejemplo en combinación con `map`

```
valores = [1, 2, 3, 4]
cuadrados = list(map(lambda x: x**2, valores))
print(cuadrados) # [1, 4, 9, 16]
```

★ Útiles para funciones simples y de una sola expresión.

8 MÓDULOS E IMPORTACIÓN

Puedes usar funciones definidas en otros archivos (.py).

módulo: operaciones.py	en otro archivo: main.py
<pre>def suma(a, b): return a + b def resta(a, b): return a - b</pre>	<pre>import operaciones as op print(op.suma(3, 4)) # 7 from operaciones import suma print(suma(5, 2)) # 7</pre>

TIP: Usa nombres cortos y claros para los módulos.

9 FUNCIÓN PRINCIPAL

Usa el bloque `if __name__ == "__main__":` para ejecutar código solo cuando el script se ejecuta directamente.

```
def main():
    print("Programa principal")

if __name__ == "__main__":
    main()

# Si este archivo se importa como módulo,
# el código de arriba NO se ejecuta.
```

10 BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos para funciones y variables.
- ✓ Cada función debe hacer una sola cosa.
- ✓ Escribe docstrings para documentar tu código.
- ✓ Evita funciones demasiado largas.
- ✓ Prueba tus funciones con casos simples primero.
- ✓ Reutiliza funciones para evitar repetir código.
- ✓ Mantén tu código limpio e indentado correctamente.



11 ERRORES FRECUENTES

- ✗ Olvidar los dos puntos (:) al definir funciones.
- ✗ Indentación incorrecta.
- ✗ Usar `return` fuera de una función.
- ✗ Confundir parámetros y argumentos.
- ✗ Modificar variables globales sin usar `global`.
- ✗ Olvidar importar módulos antes de usarlos.
- ✗ No usar docstrings ni comentarios.

★ RECUERDA

- Los scripts organizan y ejecutan programas completos.
- Las funciones permiten reutilizar y modularizar el código.
- Python es sensible a la indentación.
- Usa `import` para aprovechar el ecosistema de bibliotecas.
- ¡Código claro, funciones pequeñas y bien documentadas!

definir → reutilizar → probar → mejorar



6 ÁLGEBRA LINEAL CON NumPy Y SciPy

Herramientas para resolver sistemas, descomposiciones, valores propios y más.



Python

Lenguaje de programación para el cálculo científico.



NumPy



SciPy



Matplotlib



SymPy

1 CREAR MATRICES Y VECTORES

Importa NumPy para trabajar con arreglos.

```
import numpy as np
```

Ejemplos

```
A = np.array([1, 2, 3],
             [4, 5, 6],
             [7, 8, 9]]) # matriz 3x3

b = np.array([1, -1, 2]) # vector 1D

C = np.eye(3) # identidad 3x3

D = np.random.rand(3, 3) # aleatoria [0,1]
```



TIP: Asegúrate de que las dimensiones sean compatibles antes de operar.

2 RESOLVER SISTEMAS LINEALES

Resuelve $Ax = b$ de forma eficiente.

```
A = np.array([3, 1, -1],
             [2, 4, 1],
             [-1, 2, 5])

b = np.array([1, -1, 2])
x = np.linalg.solve(A, b)
print("Solución x =", x)
```

Salida:

```
Solución x = [ 1. -1.  1.]
```



¡IMPORTANTE!

Evita calcular la inversa explícita.
Usa `np.linalg.solve(A, b)` en su lugar.

3 DIAGNÓSTICO DE MATRICES

Funciones útiles para analizar matrices.

Función	Descripción	Ejemplo
<code>np.linalg.det(A)</code>	Determinante	<code>np.linalg.det(A)</code>
<code>np.linalg.matrix_rank(A)</code>	Rango de la matriz	<code>np.linalg.matrix_rank(A)</code>
<code>np.linalg.cond(A)</code>	Número de condición	<code>np.linalg.cond(A)</code>
<code>np.linalg.norm(A)</code>	Norma de Frobenius (por defecto)	<code>np.linalg.norm(A)</code>
<code>np.allclose(A, B)</code>	Compara matrices	<code>np.allclose(A, B)</code>

Ejemplo

```
detA = np.linalg.det(A)
rangoA = np.linalg.matrix_rank(A)
condA = np.linalg.cond(A)
normA = np.linalg.norm(A)
print(detA, rangoA, condA, normA)
```



Un número de condición muy grande indica que el sistema puede ser sensible a errores numéricos.

4 VALORES Y VECTORES PROPIOS

Calcula valores y vectores propios de A.

```
valores, vectores = np.linalg.eig(A)
print("Valores propios:", valores)
print("Vectores propios (columnas):")
print(vectores)
```

Verificación

```
lambda = valores[0]
v = vectores[:, 0]
print(np.allclose(A @ v, lambda * v)) # True
```



Para matrices simétricas/hermíticas usa `np.linalg.eigh(A)` (más eficiente y garantiza valores reales).

5 DESCOMPOSICIONES

Factoriza matrices para resolver y analizar problemas.

QR (Ortogonal - Triangular)

```
Q, R = np.linalg.qr(A)
print("Q^T Q = \n", Q.T @ Q)
print("R = \n", R)
```

Q ortogonal: $Q^T Q = I$

LU (con SciPy)

```
from scipy.linalg import lu
P, L, U = lu(A)
print("P = \n", P)
print("L = \n", L)
print("U = \n", U)
```

$PA = LU$

SVD (Valores Singulares)

```
U, s, Vh = np.linalg.svd(A)
s = np.diag(s)
print("s (singulares):", s)
print("A = U @ s @ Vh")
```

$A = U @ s @ Vh$

Cholesky (A simétrica definida pos.)

```
from scipy.linalg import cholesky
L = cholesky(A)
print("L = \n", L)
```

$A = L @ L.T$



Las descomposiciones mejoran estabilidad numérica y reducen el costo computacional.

6 PRODUCTOS Y TRANSFORMACIONES

Operaciones matriciales importantes.

Producto matricial

$C = A @ B$ # equivalente a `np.dot(A, B)`

Transpuesta y conjugada

$A.T$ # transpuesta

$A.conj().T$ # conjugada transpuesta

Traza y diagonales

`np.trace(A)` # traza

`np.diag(A)` # diagonal principal

`np.diag(np.diag(A))` # matriz diagonal



Usa `@` para producto matricial y `*` solo para producto elemento a elemento.

9 RESUMEN RÁPIDO DE FUNCIONES CLAVE

Problema	Función (NumPy/SciPy)	Descripción
Resolver $Ax = b$	<code>np.linalg.solve(A, b)</code>	Sistema lineal cuadrado.
Determinante	<code>np.linalg.det(A)</code>	Determinante de A.
Rango	<code>np.linalg.matrix_rank(A)</code>	Rango de la matriz.
Condición	<code>np.linalg.cond(A)</code>	Número de condición.
Norma	<code>np.linalg.norm(A)</code>	Norma de Frobenius (por defecto).
Valores propios	<code>np.linalg.eig(A)</code>	Valores y vectores propios.
Decomp. QR	<code>np.linalg.qr(A)</code>	Q ortogonal, R triangular.
Decomp. SVD	<code>np.linalg.svd(A)</code>	U, valores singulares, V ^H .
Decomp. LU	<code>scipy.linalg.lu(A)</code>	P, L, U tales que $PA = LU$.
Cholesky	<code>scipy.linalg.cholesky(A)</code>	Para matrices SPD.

10 ERRORES FRECUENTES

- ❌ Calcular la inversa y luego multiplicar. Usa `solve(A, b)` en su lugar.
- ❌ Confundir `*` (elemento a elemento) con `@` (matricial).
- ❌ No verificar dimensiones y compatibilidad.
- ❌ Ignorar el número de condición. Puede indicar inestabilidad.
- ❌ Usar la función incorrecta para matrices no cuadradas.
- ❌ Olvidar transponer cuando es necesario.
- ❌ Ignorar unidades y escalas de los datos.

11 BUENAS PRÁCTICAS

- ✅ Usa siempre `np.array` para trabajar con datos.
- ✅ Verifica dimensiones con `.shape` antes de operar.
- ✅ Prefiere funciones de NumPy/SciPy sobre bucles.
- ✅ Aprovecha las descomposiciones para eficiencia.
- ✅ Para matrices simétricas usa `eigh` en vez de `eig`.
- ✅ Comenta tu código y usa nombres descriptivos.
- ✅ Valida resultados con casos simples o analíticos.
- ✅ Consulta la documentación: [numpy.org](https://numpy.org/doc/) y [scipy.org](https://scipy.org/docs/).



Consejo general:

NumPy es la base del cálculo numérico en Python.

SciPy complementa con algoritmos avanzados de Álgebra Lineal.

¡Consulta la documentación oficial para conocer todas las opciones disponibles!

NumPy: numpy.org/doc/

SciPy: [docs.scipy.org/](https://scipy.org/docs/)



Curso CE1111 - Análisis Numérico para Ingeniería
Escuela de Ingeniería en Computadores



Profesor
Dr. Juan Pablo Soto Quirós



Correo
jusoto@tec.ac.cr



1 FUNCIONES ÚTILES DE NumPy

Operaciones con arreglos y funciones matemáticas.

```
import numpy as np # f(x) = x^2 - 4
x = np.array([0, np.pi/2, np.pi])
y = np.sin(x)
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A @ B
eigvals, eigvecs = np.linalg.eig(A)
norma = np.linalg.norm(A, 2) # norma 2
```

Funciones comunes

- np.linspace(a, b, n)
 - np.zeros(shape)
 - np.eye(n)
 - np.exp(x)
 - np.dot(A, B)
 - np.mean(x), np.std(x)
- np.arange(a, b, paso)
 - np.ones(shape)
 - np.sin(x), np.cos(x)
 - np.log(x), np.sqrt(x)
 - np.linalg.inv(A)
 - np.sum(x), np.prod(x)

 **TIP:** NumPy es la base para casi todas las operaciones numéricas.

2 ¿QUÉ FUNCIÓN USAR?

Función (SciPy)	¿Qué resuelve?	Entrada principal	Salida
<code>scipy.integrate.quad</code>	Integrales definidas de $f(x)$ en $[a, b]$	función f , límites a, b	valor de la integral
<code>scipy.optimize.root</code>	Raíces de ecuaciones $f(x) = 0$	función f , punto inicial x_0	raíz encontrada
<code>scipy.optimize.minimize</code>	Mínimos (o máximos) de funciones	función f , punto inicial x_0 , método	punto óptimo, valor mínimo
<code>scipy.optimize.fminbound</code>	Mínimo de una función en un intervalo $[a, b]$	función f , a, b (tolerancias opc.)	x^* que minimiza $f(x)$ en $[a, b]$
<code>scipy.linalg</code> (valores y vectores propios)	Álgebra lineal avanzada	matrices	valores y vectores propios

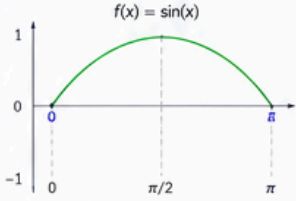
3 INTEGRACIÓN NUMÉRICA (`scipy.integrate.quad`)

Calcula $\int_a^b f(x) dx$.

```
from scipy.integrate import quad
import numpy as np

f = lambda x: np.sin(x)
I, error = quad(f, 0, np.pi)
print(I, error)
```

Resultado: 2.0 (aprox.)



 **TIP:** Funciona con funciones escalares.

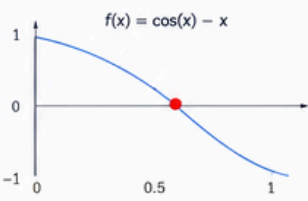
4 RESOLVER SISTEMAS NO LINEALES (`scipy.optimize.root`)

Encuentra x tal que $f(x) = 0$.

```
from scipy.optimize import root
import numpy as np

f = lambda x: np.cos(x) - x
sol = root(f, x0=0.5)
print(sol.x) # raíz encontrada
```

Resultado: [0.73908513...]



 **TIP:** Elige un buen punto inicial x_0 .

5 OPTIMIZACIÓN (`scipy.optimize.minimize`)

Minimiza una función $f(x)$.

```
from scipy.optimize import minimize
import numpy as np

f = lambda x: (x-2)**2 + 1
res = minimize(f, x0=0)
print(res.x) # mínimo
print(res.fun) # valor mínimo
```

Resultado: [2.] 1.0



 **TIP:** Métodos: 'BFGS', 'Nelder-Mead', 'CG', 'Powell', etc.

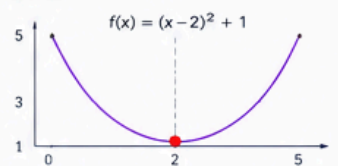
6 MÍNIMO EN UN INTERVALO (`scipy.optimize.fminbound`)

Busca el mínimo de $f(x)$ en $[a, b]$.

```
from scipy.optimize import fminbound
import numpy as np

f = lambda x: (x-2)**2 + 1
xmin = fminbound(f, 0, 6)
fmin = f(xmin)
print(xmin) # x que minimiza
print(fmin) # valor mínimo
```

Resultado:
2.0000
1.0000



 **TIP:** Útil cuando el mínimo está dentro de un intervalo acotado.

7 SISTEMAS LINEALES (`scipy.linalg.solve`)

Resuelve $Ax = b$

```
from scipy.linalg import solve
import numpy as np

A = np.array([[3, 1], [1, 2]])
b = np.array([1, 8])
x = solve(A, b)
print(x)
```

Resultado: [2. 3.]

 **TIP:** Usa solve para sistemas no singulares.

8 VALORES Y VECTORES PROPIOS (`scipy.linalg.eig`)

Calcula valores y vectores propios de A .

```
from scipy.linalg import eig
import numpy as np

A = np.array([[1, 2], [2, 1]])
vals, vecs = eig(A)
print(vals) # valores propios
print(vecs) # vectores propios
```

Resultado:
[3. -1.]
[[0.70710678 -0.70710678]
 [0.70710678 0.70710678]]

 **TIP:** w son valores propios; columnas de v son vectores propios.

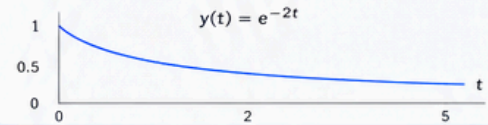
9 ECUACIONES DIFERENCIALES (`scipy.integrate.solve_ivp`)

Resuelve problemas de valor inicial y $f(t, y)$.

```
from scipy.integrate import solve_ivp
import numpy as np

def f(t, y):
    return -2*y

sol = solve_ivp(f, [0, 5], [1], t_eval=np.linspace(0, 5, 100))
print(sol.t[5]) # tiempos
print(sol.y[:,0][5]) # solución
```



 **TIP:** Ideal para EDOs y modelos dinámicos.

BUENAS PRÁCTICAS

- ✓ Usa funciones de SciPy en lugar de implementar métodos desde cero.
- ✓ Elige un buen punto inicial o intervalo según el problema.
- ✓ Verifica resultados con casos simples conocidos.
- ✓ Usa NumPy para manipular datos y SciPy para resolver problemas.
- ✓ Consulta la documentación oficial: docs.scipy.org

ERRORES FRECUENTES

- ✗ Olvidar pasar el punto inicial (x_0) en root o minimize.
- ✗ Usar métodos inadecuados para el problema.
- ✗ Elegir un intervalo incorrecto en fminbound.
- ✗ No revisar si la solución converge (sol.success).
- ✗ Ignorar unidades y escalas de los datos.

- `sol.success`

→ ¿Convergió?
- `sol.x`

→ Solución encontrada
- `sol.fun`

→ Valor de la función en el óptimo
- `sol.message`

→ Mensaje del algoritmo

 **Consejo general:**

NumPy maneja los datos y las operaciones básicas.
SciPy proporciona algoritmos avanzados para resolver problemas científicos.
¡Explora la documentación oficial para descubrir todo su potencial!

 Documentación SciPy:
<https://docs.scipy.org/>



1 IMPORTAR SymPy

Debe cargarse al iniciar la sesión.

```
import sympy as sp
sp.init_printing() # salida bonita
```



TIP: Usa `sp.init_printing()` para mejorar la presentación de las expresiones.

2 SÍMBOLOS Y EXPRESIONES

Crea símbolos y construye expresiones.

```
x, y, z = sp.symbols('x y z')
a = sp.symbols('a', real=True)
f = x**2 + sp.sin(y) + z
g = sp.exp(x) * sp.log(y)
```



Los símbolos no tienen valor numérico hasta que se les asigne uno.

3 EXPRESIONES PREDEFINIDAS

Funciones matemáticas comunes.

```
f = sp.sin(x) + sp.cos(x)
g = sp.exp(x) + sp.log(x)
h = sp.sqrt(x**2 + 1)
k = sp.Abs(x)
```

Constantes útiles

`sp.pi`, `sp.E`, `sp.I`, `sp.oo`, `sp.nan`

4 CONVERSIÓN ENTRE TIPOS

Numérico ↔ Simbólico.

```
f_num = sp.N(sp.pi)
f_sym = sp.sympify('x**2 + 1')
f_sym2 = sp.S(2)/3 # racional exacta
```



Usa `sp.N(expr)` para evaluación numérica aproximada.

5 SUSTITUCIÓN Y EVALUACIÓN

Sustituye valores simbólicos.

Una variable

```
expr = x**2 + 2*x + 1
expr.subs(x, 3) # 16
```

Varias variables

```
expr = x*y + sp.sin(z)
expr.subs({x: 2, y: 3, z: sp.pi/2}) # 7
```

Evaluación numérica

```
expr.evalf() # evalúa a float
expr.evalf(10) # 10 dígitos
```



Usa `subs` para evaluaciones exactas; `evalf` para valores aproximados.

6 DERIVADAS

Cálculo de derivadas simbólicas.

Primera derivada

```
f = sp.sin(x)*sp.exp(x)
df = sp.diff(f, x)
df # exp(x)*(sin(x) + cos(x))
```

Segunda derivada

```
d2f = sp.diff(f, x, 2)
d2f # exp(x)*(2*cos(x))
```

Derivadas parciales

```
g = x**2*y + sp.sin(x*y)
sp.diff(g, x) # 2*x*y + y*cos(x*y)
sp.diff(g, y) # x**2 + x*cos(x*y)
```



`diff(expr, var, n)` deriva `n` veces respecto a `var`.

7 INTEGRALES

Integra expresiones simbólicas.

Integral indefinida

```
f = sp.sin(x)
F = sp.integrate(f, x)
F # -cos(x)
```

Integral definida

```
I = sp.integrate(sp.sin(x), (x, 0, sp.pi))
I # 2
```

Integrales múltiples

```
g = x*y
I2 = sp.integrate(g, (x, 0, 1), (y, 0, 2))
I2 # 1
```



Usa la tupla `(var, a, b)` para límites de integración.

8 LÍMITES Y SERIES

Límites

```
sp.limit(sp.sin(x)/x, x, 0) # 1
sp.limit((1 + 1/x)**x, x, sp.oo) # E
```

Serie de Taylor

```
f = sp.exp(x)
sp.series(f, x, 0, 5)
# 1 + x + x**2/2 + x**3/6 + x**4/24 + O(x**5)
```

Expansión (polinómica)

```
sp.expand((x + 1)**3)
# x**3 + 3*x**2 + 3*x + 1
```



`series(expr, var, a, n)` expande alrededor de `a` con `n` términos.

9 RESOLUCIÓN DE ECUACIONES

Ecuaciones algebraicas.

Ecuaciones

```
sp.solve(x**2 - 5*x + 6, x)
# [2, 3]
```

Sistema de ecuaciones

```
sp.solve([x + y - 3, x - y - 1], [x, y])
# {x: 2, y: 1}
```

Ecuaciones simbólicas

```
sp.solve(a*x - b, x)
# b/a
```



Para sistemas, pasa una lista de ecuaciones

10 SIMPLIFICACIÓN

Simplifica expresiones.

```
expr = sp.sin(x)**2 + sp.cos(x)**2
sp.simplify(expr) # 1
sp.expand((x+1)*(x-1)) # x**2 - 1
sp.factor(x**2 - 1) # (x - 1)*(x + 1)
sp.cancel((x**2 - 1)/(x - 1)) # x + 1
sp.together(1/(x) + 1/(x+1))
# (2*x + 1)/(x*(x + 1))
```



`simplify` aplica varias técnicas automáticamente.

11 MATRICES SIMBÓLICAS

Operaciones con matrices.

```
A = sp.Matrix([[1, x], [0, 1]])
B = sp.Matrix([[x, 2], [3, 4]])
A + B # suma
A * B # multiplicación
A.det() # determinante -> 1
A.inv() # inversa simbólica
A**n # potencia
A.eigenvals() # autovalores
```



Usa `Matrix` para trabajar con matrices simbólicas.

12 DESIGUALDADES

Resolver desigualdades.

```
sp.solve_univariate_inequality(x**2 - 4 > 0, x)
# (-oo, -2) U (2, oo)
sp.reduce_inequalities([x - 1 > 0, x + 2 < 3], x)
# (1 < x) & (x < 1)
# (equivala a intervalo (1, 1))
```



`solve_univariate`.

13 RESUMEN DE COMANDOS CLAVE DE SymPy

Comando	¿Qué hace?	Ejemplo	Resultado
<code>symbols()</code>	Crea símbolos	<code>sp.symbols('x y')</code>	<code>x, y</code>
<code>sympify()</code>	Convierte a expresión simbólica	<code>sp.sympify('x**2 + 1')</code>	<code>x**2 + 1</code>
<code>S()</code>	Número exacto (racional)	<code>sp.S(1)/3</code>	<code>1/3</code>
<code>sin(), cos(), exp(), log(), sqrt()</code>	Funciones matemáticas	<code>sp.sin(x)</code> , <code>sp.exp(x)</code>	<code>sin(x)</code> , <code>exp(x)</code>
<code>subs()</code>	Sustitución	<code>expr.subs(x, 2)</code>	valor exacto
<code>diff()</code>	Derivadas	<code>sp.diff(x**2, x)</code>	<code>2*x</code>
<code>integrate()</code>	Integrales	<code>sp.integrate(sp.sin(x), x)</code>	<code>-cos(x)</code>
<code>limit()</code>	Límites	<code>sp.limit(sp.sin(x)/x, x, 0)</code>	<code>1</code>
<code>series()</code>	Serie de Taylor	<code>sp.series(sp.exp(x), x, 0, 4)</code>	<code>1 + x + x**2/2 + x**3/6 + ...</code>
<code>solve()</code>	Ecuaciones	<code>sp.solve(x**2 - 1, x)</code>	<code>[-1, 1]</code>
<code>simplify()</code>	Simplificación general	<code>sp.simplify(sp.sin(x)**2 + sp.cos(x)**2)</code>	<code>1</code>
<code>Matrix()</code>	Crea matrices	<code>sp.Matrix([[1, 2], [3, 4]])</code>	<code>Matrix([[1, 2], [3, 4]])</code>
<code>Eq()</code>	Ecuación simbólica	<code>sp.Eq(x + 1, 0)</code>	<code>Eq(x + 1, 0)</code>
<code>factor()</code>	Factorización	<code>sp.factor(x**2 - 1)</code>	<code>(x - 1)*(x + 1)</code>

14 BUENAS PRÁCTICAS

- ✓ Usa símbolos en lugar de números para obtener resultados generales.
- ✓ Simplifica expresiones antes y después de derivar o integrar.
- ✓ Verifica supuestos (positividad, enteros, etc.) cuando sea necesario:
`x = sp.symbols('x', positive=True)`
- ✓ Usa `sp.N(expr, n)` para obtener aproximaciones numéricas con `n` dígitos.
- ✓ Combina SymPy con NumPy para evaluaciones numéricas vectorizadas.

15 ERRORES FRECUENTES

- ✗ Olvidar importar SymPy: `import sympy as sp`.
- ✗ Confundir `**` (potencia) con `^` (XOR en Python).
- ✗ No definir símbolos antes de usarlos.
- ✗ No simplificar expresiones complicadas.
- ✗ Esperar resultados numéricos cuando todo es simbólico.



Consejo general:

SymPy complementa a NumPy y SciPy proporcionando cálculos exactos y manipulaciones algebraicas. ¡Explora la documentación oficial para descubrir todo su potencial!



Documentación SymPy:
<https://docs.sympy.org/>



1 IMPORTAR Y PREPARAR

Importa la librería y prepara el entorno.

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline # en Jupyter
```

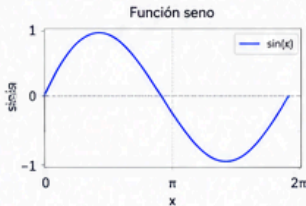


TIP: Usa `%matplotlib inline` en Jupyter para mostrar gráficas dentro del cuaderno.

2 GRÁFICO BÁSICO CON plot()

Gráfica datos y personaliza estilo.

```
x = np.linspace(0, 2*np.pi, 400)
y = np.sin(x)
plt.plot(x, y, 'b-', label='sin(x)')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Función seno')
plt.legend()
plt.grid(True)
plt.show()
```

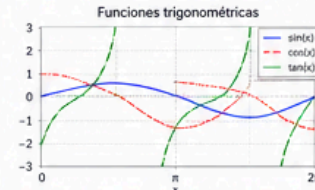


TIP: `label` muestra la leyenda cuando usas `plt.legend()`.

3 MÚLTIPLES GRÁFICAS

Dibuja varias curvas en la misma figura.

```
x = np.linspace(0, 2*np.pi, 400)
plt.plot(x, np.sin(x), 'b', label='sin(x)')
plt.plot(x, np.cos(x), 'r--', label='cos(x)')
plt.plot(x, np.tan(x), 'g--', label='tan(x)')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Funciones trigonométricas')
plt.legend(loc='best')
plt.grid(True, ls='--', alpha=0.6)
plt.xlim(0, 2*np.pi)
plt.ylim(-3, 3)
plt.show()
```



TIP: `xlim()`, `ylim()` controlan los límites de los ejes.

4 ESTILOS DE LÍNEA Y COLORES

Especifica colores, estilos y marcadores.

Color	Ejemplo
'b'	azul
'r'	rojo
'g'	verde
'k'	negro
'm'	magenta
'c'	cian
'y'	amarillo

Estilos de línea	Ejemplo
'-'	línea sólida
'--'	línea discontinua
'-.'	línea punto-guion
':'	línea punteada

Marcadores	Ejemplo
'o'	círculo
's'	cuadrado
'^'	triángulo arriba
'*'	estrella



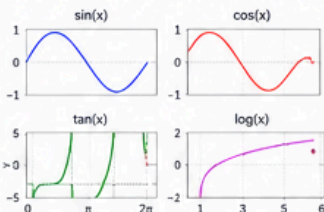
TIP: Combina color + estilo + marcador. Ej.: `'ro--'`, `'bs--'`, `'g*'`.

5 SUBGRÁFICAS (subplot)

Crea varias gráficas en una figura.

```
x = np.linspace(0, 2*np.pi, 400)
fig, axs = plt.subplots(2, 2, figsize=(8,6))

axs[0,0].plot(x, np.sin(x), 'b')
axs[0,0].set_title('sin(x)')
axs[0,1].plot(x, np.cos(x), 'r')
axs[0,1].set_title('cos(x)')
axs[1,0].plot(x, np.tan(x), 'g')
axs[1,0].set_title('tan(x)')
axs[1,1].plot(x, np.log(x), 'm')
axs[1,1].set_title('log(x)')
for ax in axs.flat:
    ax.grid(True, ls='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

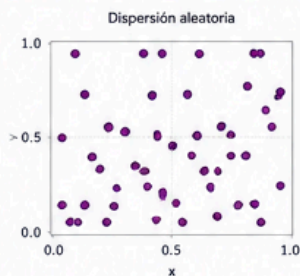


TIP: `plt.subplots(filas, columnas)` organiza mejor tus gráficas.

6 SCATTER Y DISPERSIÓN

Gráfica puntos individuales.

```
np.random.seed(0)
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y, c='purple', marker='o',
            s=60, edgecolors='k')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Dispersión aleatoria')
plt.grid(True, ls='--', alpha=0.6)
plt.show()
```

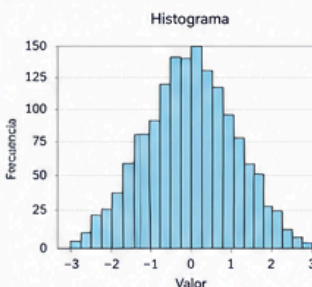


TIP: `s` controla el tamaño y `c` el color de los puntos.

7 HISTOGRAMA

Visualiza la distribución de datos.

```
np.random.seed(1)
datos = np.random.randn(1000)
plt.hist(datos, bins=25, color='skyblue',
         edgecolor='k', alpha=0.8)
plt.xlabel('Valor')
plt.ylabel('Frecuencia')
plt.title('Histograma')
plt.grid(True, axis='y', ls='--', alpha=0.6)
plt.show()
```

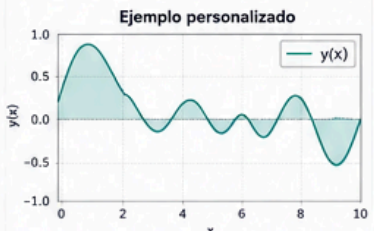


TIP: `bins` define el número de barras del histograma.

8 PERSONALIZACIÓN AVANZADA

Mejora la apariencia de tus gráficas.

```
x = np.linspace(0, 10, 400)
y = np.exp(-0.3*x) * np.cos(2*x)
plt.figure(figsize=(7,4))
plt.plot(x, y, 'teal', lw=2.5, label='y(x)')
plt.fill_between(x, y, 0, color='teal',
                alpha=0.2)
plt.axhline(0, color='gray', lw=1, ls='--')
plt.xlabel('x', fontsize=12)
plt.ylabel('y(x)', fontsize=12)
plt.title('Ejemplo personalizado',
        fontsize=14, weight='bold')
plt.legend(frameon=True, fancybox=True)
plt.grid(True, ls=':', alpha=0.7)
plt.tight_layout()
plt.show()
```



TIP: Usa `figsize`, `lw`, `alpha`, `grid`, `legend` para gráficas profesionales.

9 MÉTODOS ÚTILES DE pyplot

Funciones y métodos más usados.

Método	Descripción	Método	Descripción
<code>plt.plot()</code>	Gráfica de líneas	<code>plt.xlim()</code> , <code>plt.ylim()</code>	Límites de los ejes
<code>plt.scatter()</code>	Gráfica de dispersión	<code>plt.grid()</code>	Muestra cuadrícula
<code>plt.hist()</code>	Histograma	<code>plt.axhline()</code> , <code>plt.axvline()</code>	Líneas horizontales/verticales
<code>plt.bar()</code>	Gráfica de barras	<code>plt.text()</code>	Agregar texto en la gráfica
<code>plt.pie()</code>	Gráfico de pastel	<code>plt.tight_layout()</code>	Ajusta espacios automáticamente
<code>plt.xlabel()</code> , <code>plt.ylabel()</code>	Etiquetas de ejes	<code>plt.savefig('fig.png')</code>	Guarda la figura como imagen
<code>plt.title()</code>	Título de la gráfica	<code>plt.show()</code>	Muestra la gráfica en pantalla
<code>plt.legend()</code>	Muestra la leyenda		

10 COLORES Y ESTILOS RÁPIDOS

Colores y estilos predefinidos.

Paletas de colores	Estilo global
'viridis' 'plasma' 'inferno' 'magma' 'coolwarm' 'Greys'	<pre>import matplotlib.pyplot as plt plt.style.use('seaborn-v0.8') # estilos</pre> Estilos populares: 'default' 'ggplot' 'dark_background' 'seaborn-v0.8' 'fivethirtyeight'



TIP: Cambiar el estilo global mejora la apariencia con una sola línea.

11 BUENAS PRÁCTICAS

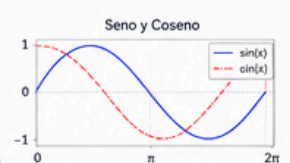
- ✓ Siempre etiqueta los ejes y agrega título.
- ✓ Usa leyendas claras para distinguir curvas.
- ✓ Activa la cuadrícula para mejor lectura.
- ✓ Evita saturar la gráfica con demasiados elementos.
- ✓ Guarda tus figuras con `plt.savefig()` para reportes.
- ✓ Usa tamaños y colores consistentes.

12 ERRORES FRECUENTES

- ✗ Olvidar `plt.show()` (la gráfica no se muestra).
- ✗ No cerrar figuras en bucles (usa `plt.close()`).
- ✗ Etiquetas de ejes o leyendas confusas.
- ✗ Usar demasiados colores o estilos diferentes.
- ✗ No controlar los límites de los ejes.

Ejemplo rápido completo

```
x = np.linspace(0, 2*np.pi, 400)
plt.figure(figsize=(6,4))
plt.plot(x, np.sin(x), 'b', label='sin(x)')
plt.plot(x, np.cos(x), 'r--', label='cos(x)')
plt.plot(x, np.tan(x), 'g--', label='tan(x)')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.title('Seno y Coseno')
plt.legend(); plt.grid(True, ls='--', alpha=0.6)
plt.tight_layout(); plt.show()
```



Consejo general: Matplotlib es muy potente y flexible. ¡Explora la documentación oficial para descubrir todo su potencial!



Documentación Matplotlib:
<https://matplotlib.org/stable/>



Cálculo numérico con arreglos

✓ BUENAS PRÁCTICAS

- ✓ Use arreglos de NumPy en lugar de listas.
`x = np.array([1, 2, 3])`
mejor que `[1, 2, 3]`
- ✓ Especifique dtype cuando sea necesario.
`A = np.array([1, 2, 3], dtype=float)`
- ✓ Use operaciones vectorizadas en lugar de bucles.
`y = x**2 + 2*x + 1`
no usar `for`
- ✓ Preasigne memoria en bucles grandes.
`n = 1000`
`y = np.zeros(n)`
`for i in range(n):`
`y[i] = i**2`
- ✓ Use funciones de NumPy para estadísticas.
`m = np.mean(x)`
`s = np.std(x)`

✗ ERRORES FRECUENTES

- ✗ Mezclar listas y arreglos.
`x = [1, 2, 3]`
`y = np.array([1, 2, 3])`
`x + y` # Error
- ✗ Usar bucles innecesarios (lento).
`y = []`
`for i in x:`
`y.append(i**2)` # lento
- ✗ Olvidar el operador `.` para funciones y métodos.
`A = np.array([1, 2, 3])`
`A.mean` # Error
`A.mean()` # Correcto
- ✗ Confundir `*` (elemento a elemento) con `@` (producto matricial).
`A * B` # elemento a elemento
`A @ B` # producto matricial
- 5 Cambios de dtype inesperados.
`a = np.array([1, 2])`
`b = np.array([1.5, 2.5])`
`a + b` # Resultado float

💡 TIP: Revise siempre `shape` y `dtype`: `x.shape`, `x.dtype`



Algoritmos científicos y matemáticos

✓ BUENAS PRÁCTICAS

- ✓ Lea la documentación antes de usar un método.
`help(scipy.optimize.root)`
- ✓ Proporcione un buen punto inicial.
`sol = root(f, x0=[1, 1])`
- ✓ Establezca tolerancias adecuadas.
`root(f, x0, tol=1e-10)`
- ✓ Verifique la convergencia de los resultados.
`sol = root(f, x0)`
`if sol.success:`
`x = sol.x`
- ✓ Aproveche funciones específicas del problema.
`solve_ivp()` para EDO
`minimize()` para óptimos

✗ ERRORES FRECUENTES

- ✗ No verificar si el algoritmo convergió.
`sol = root(f, x0)`
`x = sol.x` # Puede fallar
- ✗ Usar métodos no adecuados para el problema.
Ejemplo:
`root()` para minimización
- ✗ Ignorar unidades o escalas del problema.
Puede causar mal condicionamiento
- ✗ Solicitar demasiada precisión innecesaria.
`tol = 1e-16` # Puede no alcanzarse
- ✗ No manejar excepciones.
`try:`
`sol = minimize(f, x0)`
`except Exception as e:`
`print(e)`

💡 TIP: Revise `sol.success`, `sol.message` y `sol.nfev`



Cálculo simbólico exacto

✓ BUENAS PRÁCTICAS

- ✓ Use símbolos en lugar de números literales.
`x = sp.symbols('x')`
`f = x**2 + 1`
- ✓ Simplifique expresiones.
`sp.simplify(f)`
- 3 Use `sp.Eq` para ecuaciones.
`sp.solve(sp.Eq(x**2, 4), x)`
- 4 Evalúe numéricamente con `lambdify` cuando sea posible.
`f_num = sp.lambdify(x, f)`
`f_num(2.0)`
- 5 Imprima expresiones de forma legible.
`sp.init_printing()`

✗ ERRORES FRECUENTES

- ✗ Mezclar SymPy con NumPy sin convertir.
`x = sp.symbols('x')`
`np.sin(x)` # Error
- 2 Usar `==` en lugar de `sp.Eq` para ecuaciones.
`sp.solve(x**2 == 4, x)`
Incorrecto
- 3 No simplificar expresiones grandes.
`expr.expand()` # Puede hacerlo más grande
- 4 Convertir a float antes de tiempo.
`float(expr)` # Pierde exactitud
- 5 No definir símbolos como reales cuando aplica.
`x = sp.symbols('x', real=True)`

💡 TIP: `sp.simplify()`, `sp.factor()`, `sp.pretty()`



Visualización de datos

✓ BUENAS PRÁCTICAS

- ✓ Use `plt.figure()` antes de crear gráficos.
`plt.figure(figsize=(6, 4))`
- ✓ Siempre etiquete ejes, título y leyenda.
`plt.xlabel('x')`
`plt.ylabel('y')`
`plt.title('Titulo')`
- ✓ Use grid para mejorar la lectura.
`plt.grid(True, ls='--', alpha=0.7)`
- ✓ Guarde figuras con dpi adecuado.
`plt.savefig('fig.png', dpi=300, bbox_inches='tight')`
- 5 Cierre figuras que ya no usa.
`plt.close()`

✗ ERRORES FRECUENTES

- ✗ No llamar `plt.show()` al final (no aparece nada).
`plt.plot(x, y)`
Falta `plt.show()`
- ✗ Crear demasiadas figuras sin cerrar.
`for i in range(100):`
`plt.figure()` # Consume RAM
- ✗ No ajustar límites adecuadamente.
Gráfica poco clara
`plt.plot(x, y)`
- ✗ Usar demasiados colores o estilos.
Dificulta la lectura
- ✗ No usar leyenda cuando hay varias curvas.
`plt.plot(x, y1)`
`plt.plot(x, y2)`
Sin `plt.legend()`

💡 TIP: `plt.tight_layout()` mejora la distribución



BUENAS PRÁCTICAS GENERALES

- ✓ Comente su código de forma clara.
- ✓ Use nombres descriptivos para variables y funciones.
- ✓ Evite "números mágicos", use variables con significado.

- ✓ Valide entradas y revise resultados.
- ✓ Modularice su código en funciones.
- ✓ Revise y pruebe su código frecuentemente.

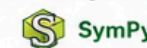
✗ ERRORES GENERALES FRECUENTES

- ✗ Copiar y pegar sin entender el código.
- ✗ No revisar documentación oficial.
- ✗ No verificar unidades y dimensiones.
- ✗ Suponer que el resultado es correcto sin validarlo.
- ✗ Ignorar advertencias y mensajes de error.
- ✗ No hacer respaldos ni control de versiones.



Consejo: La práctica constante, la lectura de documentación y la organización son claves para programar mejor.





1 VARIABLES Y TIPOS DE DATOS



Función / Tipo	Descripción
<code>int</code>	Números enteros.
<code>float</code>	Números decimales.
<code>complex</code>	Números complejos ($a + bj$).
<code>bool</code>	Valores lógicos: True o False.
<code>str</code>	Cadenas de texto.
<code>type(x)</code>	Devuelve el tipo de la variable x.
<code>print(x)</code>	Muestra el valor de x en pantalla.
<code>input(msg)</code>	Lee entrada desde el teclado.



TIP: Usa `print(type(x))` para verificar el tipo de una variable.

2 LISTAS Y ARREGLOS NumPy



Función	Descripción
<code>list()</code>	Crea una lista.
<code>append(x)</code>	Agrega un elemento al final.
<code>len(lista)</code>	Devuelve la longitud de la lista.
<code>np.array(obj)</code>	Convierte una lista u objeto en arreglo NumPy.
<code>tolist()</code>	Convierte un arreglo NumPy a lista de Python.
<code>np.asarray(obj)</code>	Convierte sin copiar si es posible.
<code>np.copy(A)</code>	Copia explícita de un arreglo.



TIP: Los arreglos NumPy son más rápidos y eficientes que las listas para cálculos.

3 CREACIÓN DE VECTORES Y MATRICES



Función	Descripción
<code>np.zeros(shape)</code>	Matriz de ceros.
<code>np.ones(shape)</code>	Matriz de unos.
<code>np.eye(n)</code>	Matriz identidad $n \times n$.
<code>np.arange(a, b, h)</code>	Rango con paso h (similar a range).
<code>np.linspace(a, b, n)</code>	n puntos igualmente espaciados en $[a, b]$.
<code>np.logspace(a, b, n)</code>	n puntos espaciados logaritmicamente 10^a a 10^b .
<code>np.random.rand(m, n)</code>	Aleatorios uniformes en $[0, 1)$.
<code>np.random.randn(m, n)</code>	Aleatorios normales estándar.
<code>np.random.randint(1, u, s)</code>	Enteros aleatorios en $[1, u)$.



TIP: shape puede ser $(n,)$, (m, n) o $(m, n, p, \dots)$.
Ej.: `np.zeros((3,4))` crea 3 filas y 4 columnas.

4 INFORMACIÓN Y MANIPULACIÓN DE ARREGLOS



Función / Atributo	Descripción
<code>A.shape</code>	Tupla con el tamaño de cada dimensión.
<code>A.ndim</code>	Número de dimensiones.
<code>A.size</code>	Número total de elementos.
<code>A.dtype</code>	Tipo de datos de los elementos.
<code>A.reshape(n, m, ...)</code>	Cambia la forma del arreglo.
<code>A.ravel()</code>	Devuelve un vector 1D (vista).
<code>A.flatten()</code>	Devuelve un vector 1D (copia).
<code>A.T</code>	Transpuesta (invierte filas y columnas).



TIP: Usa `reshape` para cambiar la forma sin modificar los datos (si es posible).

5 INDEXACIÓN Y REBANADO



Expresión	Descripción
<code>A[i, j]</code>	Elemento en fila i y columna j.
<code>A[i, :]</code>	Fila i completa.
<code>A[:, j]</code>	Columna j completa.
<code>A[i1:i2, j1:j2]</code>	Bloque de submatriz.
<code>A[-1, :]</code>	Última fila.
<code>A[:, -1]</code>	Última columna.
<code>A[a:b]</code>	Rebanado de un vector.
<code>A[:, :k]</code>	Rebanado con paso k.



TIP: Los índices comienzan en 0. Los negativos cuentan desde el final.

6 OPERADORES Y OPERACIONES FRECUENTES

Operador / Función	Descripción
<code>A + B</code>	Suma elemento a elemento.
<code>A - B</code>	Resta elemento a elemento.
<code>A * B</code>	Multiplicación elemento a elemento.
<code>A / B</code>	División elemento a elemento.
<code>A ** k</code>	Potencia elemento a elemento.
<code>A @ B</code>	Producto matricial (Python 3.5+).
<code>np.dot(A, B)</code>	Producto punto o matricial.
<code>np.sum(A, axis=None)</code>	Suma de elementos.
<code>np.mean(A, axis=None)</code>	Promedio de elementos.
<code>np.max(A, axis=None)</code>	Valor máximo.
<code>np.min(A, axis=None)</code>	Valor mínimo.



TIP: Usa `@` o `np.dot()` para multiplicación de matrices.
* es solo para producto elemento a elemento.

7 CONDICIONALES Y CICLOS

Condicionales	Ciclos
<code>if</code> condición: instrucciones	<code>for i in range(n):</code> instrucciones
<code>elif</code> otra_condición: instrucciones	<code>while</code> condición: instrucciones
<code>else:</code> instrucciones	<code>break</code> <code>continue</code>
Funciones útiles	
<code>range(inicio, fin, paso)</code>	Genera secuencia de números.
<code>enumerate(iterable)</code>	Devuelve índice y valor.
<code>zip(a, b)</code>	Combina elementos por posición.



TIP: Usa `break` para salir del ciclo.
`continue` para saltar a la siguiente iteración.

8 ATAJS O COMANDOS MÁS UTILIZADOS

<code>print()</code>	<code>type()</code>	<code>len()</code>	<code>1 2 3</code> <code>range()</code>
<code>np.array()</code>	<code>np.zeros()</code>	<code>np.ones()</code>	<code>I</code> <code>np.eye()</code>
<code>A.shape</code>	<code>A.T</code>	<code>A @ B</code>	$\sum$ <code>np.sum()</code>
$\bar{x}$ <code>np.mean()</code>	$\uparrow$ <code>np.max()</code>	$\downarrow$ <code>np.min()</code>	<code>?</code> <code>help()</code>



TIP: `help(función)` muestra la ayuda de cualquier función o módulo.





1 SCRIPTS Y FUNCIONES



Función / Sentencia	Descripción
<code>def nombre(args):</code>	Define una función.
<code>return valor</code>	Devuelve un resultado.
<code>lambda args: expr</code>	Función anónima (lambda).
<code>import modulo</code>	Importa un módulo.
<code>from modulo import f</code>	Importa funciones específicas.
<code>as</code>	Asigna un alias al importar.
<code>if __name__ == "__main__":</code>	Ejecuta código solo si el script se ejecuta directamente.



TIP: Divide tu código en funciones y módulos para reutilizarlo y mantenerlo organizado.

2 ÁLGEBRA LINEAL



Función	Descripción
<code>np.linalg.solve(A, b)</code>	Resuelve $Ax = b$.
<code>np.linalg.det(A)</code>	Determinante de A.
<code>np.linalg.matrix_rank(A)</code>	Rango de la matriz.
<code>np.linalg.cond(A)</code>	Número de condición.
<code>np.linalg.norm(v, ord)</code>	Norma de un vector o matriz.
<code>np.linalg.eig(A)</code>	Valores y vectores propios.
<code>np.linalg.eigh(A)</code>	Para matrices simétricas/hermiticas.
<code>np.linalg.qr(A)</code>	Factorización QR.
<code>np.linalg.svd(A)</code>	Descomposición SVD.
<code>scipy.linalg.lu(A)</code>	Factorización LU.
<code>scipy.linalg.cholesky(A)</code>	Factorización de Cholesky.



TIP: Prefiere `solve(A, b)` en lugar de `inv(A) @ b` para mayor estabilidad numérica.

3 MÉTODOS NUMÉRICOS CON SciPy



Función	Descripción
<code>scipy.integrate.quad(f, a, b)</code>	Integra $f(x)$ en $[a, b]$.
<code>scipy.optimize.root(f, x0)</code>	Encuentra raíz de $f(x) = 0$.
<code>scipy.optimize.minimize(f, x0)</code>	Minimiza $f(x)$ (multivariado).
<code>scipy.optimize.fminbound(f, a, b)</code>	Minimiza $f(x)$ en $[a, b]$ (1D).
<code>scipy.integrate.solve_ivp(fun, t_span, y0)</code>	Resuelve EDOs (valor inicial).

Métodos comunes

`root:` method = 'hybr', 'lm', 'broyden1', ...
`minimize:` method = 'BFGS', 'Nelder-Mead', 'CG', ...
`solve_ivp:` method = 'RK45', 'Radau', 'BDF', ...



TIP: Revisa `sol.success`, `sol.x` y `sol.message` para validar la solución obtenida.

4 CÁLCULO SIMBÓLICO CON SymPy



Función	Descripción
<code>sp.symbols('x y z')</code>	Crea símbolos.
<code>sp.sympify(expr)</code>	Convierte a expresión simbólica.
<code>sp.S(valor)</code>	Número exacto (racional).
<code>expr.subs(x, valor)</code>	Sustituye valores.
<code>sp.diff(expr, x, n)</code>	Derivada de orden n.
<code>sp.integrate(expr, x)</code>	Integral indefinida.
<code>sp.limit(expr, x, a)</code>	Límite cuando $x \rightarrow a$.
<code>sp.series(expr, x, a, n)</code>	Serie de Taylor (orden n).
<code>sp.solve(ecuación, vars)</code>	Resuelve ecuaciones.
<code>sp.simplify(expr)</code>	Simplifica la expresión.
<code>sp.expand(expr)</code>	Expande la expresión.
<code>sp.factor(expr)</code>	Factoriza la expresión.
<code>sp.Matrix([[...]])</code>	Crea matriz simbólica.



TIP: Usa `simplify()` antes de comparar expresiones simbólicas o mostrar resultados.

5 GRÁFICAS 2D CON Matplotlib



Función	Descripción
<code>plt.figure(figsize=(w, h))</code>	Crea una nueva figura.
<code>plt.plot(x, y, 'b-', label='y1')</code>	Gráfica de línea.
<code>plt.scatter(x, y, c='r', marker='o')</code>	Gráfica de dispersión.
<code>plt.hist(data, bins=20)</code>	Histograma.
<code>plt.bar(x, height)</code>	Gráfica de barras.
<code>plt.xlabel('X')</code>	Etiqueta del eje X.
<code>plt.ylabel('Y')</code>	Etiqueta del eje Y.
<code>plt.title('Titulo')</code>	Título del gráfico.
<code>plt.legend()</code>	Muestra la leyenda.
<code>plt.grid(True)</code>	Activa la cuadrícula.
<code>plt.xlim(a, b)</code>	Límites del eje X.
<code>plt.ylim(c, d)</code>	Límites del eje Y.
<code>plt.axhline(y, ...)</code>	Línea horizontal.
<code>plt.axvline(x, ...)</code>	Línea vertical.
<code>plt.tight_layout()</code>	Ajusta el diseño.
<code>plt.savefig('fig.png', dpi=300)</code>	Guarda la figura.
<code>plt.show()</code>	Muestra la figura.
<code>plt.close()</code>	Cierra la figura actual.



TIP: Usa `label + legend()` para identificar curvas y guarda con dpi alto para reportes.

6 USOS RÁPIDOS (EJEMPLOS)



	Resolver $Ax = b$	<code>x = np.linalg.solve(A, b)</code>
	Encontrar raíz de $f(x)$	<code>from scipy.optimize import root</code> <code>sol = root(f, x0)</code>
	Calcular integral	<code>from scipy.integrate import quad</code> <code>I, err = quad(f, a, b)</code>
	Minimizar $f(x)$	<code>from scipy.optimize import minimize</code> <code>res = minimize(f, x0, method='BFGS')</code>
	Minimizar en $[a, b]$ (1D)	<code>from scipy.optimize import fminbound</code> <code>xopt = fminbound(f, a, b)</code>
	Graficar función	<code>x = np.linspace(a, b, 400)</code> <code>plt.plot(x, f(x)); plt.grid(True); plt.show()</code>



TIP: Adapta el método y parámetros a tu problema para obtener mejores resultados.

7 ERRORES FRECUENTES



- ❌ Confundir `*` (elemento a elemento) con `@` (producto matricial).
- ❌ Usar `inv(A)` para resolver sistemas en lugar de `solve(A, b)`.
- ❌ No revisar `sol.success`, `sol.message` o tolerancias en SciPy.
- ❌ Mezclar tipos NumPy y SymPy sin convertir adecuadamente.
- ❌ Olvidar que los índices en Python/NumPy comienzan en 0.
- ❌ No verificar dimensiones compatibles antes de operar matrices/arrays.
- ❌ No cerrar figuras ni usar `plt.show()` en scripts.
- ❌ Generar figuras muy grandes sin ajustar dpi o layout.
- ❌ Usar floats cuando exactitud simbólica es necesaria.
- ❌ No documentar ni comentar el código.



Consejo: Prueba tu código con casos simples antes de escalar a problemas grandes.

8 BUENAS PRÁCTICAS GENERALES



- ✅ Usa nombres descriptivos para variables y funciones.
- ✅ Divide el código en funciones pequeñas y reutilizables.
- ✅ Importa solo lo necesario.
- ✅ Comenta tu código y usa docstrings en funciones.
- ✅ Verifica entradas y dimensiones (`assert` o `try/except`).
- ✅ Prefiere funciones vectorizadas de NumPy.
- ✅ Elige métodos numéricos apropiados al problema.
- ✅ Controla la aleatoriedad con `np.random.seed(0)`.
- ✅ Guarda y versiona tu trabajo.
- ✅ Revisa y valida siempre los resultados.



Código claro + pruebas + documentación = resultados confiables y reproducibles.

